



Nagar YuwakShikshanSanstha's

Yeshwantrao Chavan College of Engineering

(An Autonomous Institution affiliated to RashttrasantTukadojiMaharaj Nagpur University)

Hingna Road, Wanadongri, Nagpur - 441 110

NAAC A++

Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu



Department of Computer Science & Engineering (IoT)

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.

Session 2026-2027

Vision: Dream of where you want.	Mission: Means to achieve Vision
---	---

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	Preparation	P: Preparation	Pep-CL abbreviation pronounce as Pep-si-IL easy to recall
PEO2	Core Competence	E: Environment (Learning Environment)	
PEO3	Breadth	P: Professionalism	
PEO4	Professionalism	C: Core Competence	
PEO5	Learning Environment	L: Breadth (Learning in diverse areas)	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:

Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

“I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life.” to contribute to the development of cutting-edge technologies and Research.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Sheikh Wasimuddin
Name of Student

**Department of Computer Science & Engineering (IoT)****Vision of the Department***To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.***Mission of the Department***To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skill through emerging technologies.*

Session	2026-27 (ODD)	Course Name	PE-V : Lab : Blockchain Technology
Semester	7	Course Code	23IOT1748
Roll No	61	Name of Student	Sheikh Wasimuddin

Practical Number	08
Course Outcome	1. Demonstrate basics of distributed systems and cryptographic primitives used in blockchain. 2. Outline various research aspects for development of blockchain. 3. Make use of different security protocols to build secure blockchain applications. 4. Analyze smart contracts and Hyperledger Fabric used in blockchain. 5. Determine the performance of applications developed with Bitcoin and Ethereum blockchain.
Aim	Implement Smart Contracts Using Mappings and Modifiers for Access Control.
Theory (100 words)	A smart contract is a blockchain-based program that automatically performs predefined operations according to the rules written in its code. Once deployed, the contract can store data and provide functions through which users can interact with the blockchain. Solidity is a programming language widely used for developing smart contracts on Ethereum-compatible blockchain networks. In Solidity, mappings are used to organize and access data through unique keys. A mapping can associate an Ethereum address with information such as a balance, role, registration status, or ownership record. This makes mappings useful for applications where information must be retrieved based on an address or another unique identifier. Modifiers provide a convenient way to apply common conditions to functions. Instead of writing the same verification code repeatedly, a modifier can contain the required condition and be attached to multiple functions. This improves code organization and reduces repetition. Access control is an important part of smart-contract security. It defines which blockchain accounts are permitted to perform specific operations. The msg.sender variable identifies the address that is currently calling a contract function. By checking this address against predefined permissions, the contract can restrict sensitive operations.

**Department of Computer Science & Engineering (IoT)****Vision of the Department***To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.***Mission of the Department***To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.*

	An owner-based control mechanism can be implemented by storing the deployer's address as the contract owner. Functions responsible for administrative activities can then be protected so that only the owner is allowed to execute them. This helps prevent unauthorized changes to important contract data. User registration and role management can also be implemented using mappings. For example, the contract can maintain a record indicating whether a particular blockchain address is registered or has a specific role. Before executing a restricted function, the contract can verify the user's status and allow or reject the operation accordingly. The <code>require()</code> statement is commonly used to validate conditions during contract execution. If the specified condition is false, the transaction is reverted. Combining mappings, modifiers, <code>msg.sender</code>, and <code>require()</code> allows Solidity developers to create structured smart contracts with controlled access and organized data management.
Procedure and Execution (100 Words)	1. Open Remix IDE and create a new Solidity file with a suitable filename. 2. Specify the required Solidity compiler version using the pragma statement. 3. Create the smart contract and declare the necessary variables for storing the contract owner and student information. 4. Create mappings to store student records and maintain the registration status of blockchain addresses. 5. Define the constructor to automatically store the address of the account that deploys the contract as the owner. 6. Create an <code>onlyOwner</code> modifier using the <code>require()</code> statement to allow only the owner to execute administrative functions. 7. Create an <code>onlyRegistered</code> modifier to verify whether the calling blockchain address has been registered. 8. Implement a function to add a student record by storing the student's ID and name in the appropriate mapping. 9. Implement a function to retrieve the stored student information using the student ID. 10. Implement an update function to modify an existing student record and use a condition to verify that the specified student is already present. 11. Implement a registration function that records the caller's address as a registered user. 12. Implement a function for transferring ownership from the current owner to another valid blockchain address. 13. Select the appropriate Solidity compiler version and compile the smart



Department of Computer Science & Engineering (IoT)

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

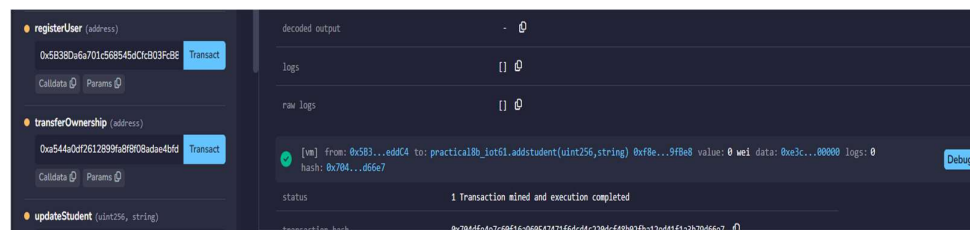
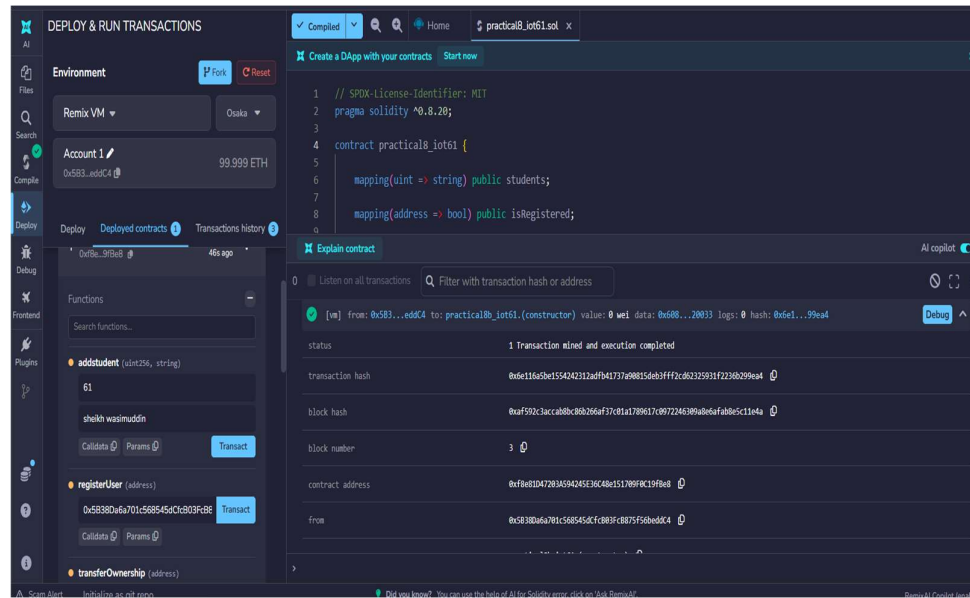
To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.

contract.

14. Open the **Deploy & Run Transactions** section in Remix and deploy the contract using the selected account.

15. Use different accounts to call the contract functions and verify the student record operations, user registration, owner-only restrictions, registered-user restrictions, and ownership transfer functionality.

Stepwise Screenshots:





Nagar YuwakShikshanSanstha's
Yeshwantrao Chavan College of Engineering
(An Autonomous Institution affiliated to RashttrasantukadojiMaharaj Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110



NAAC A++

Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu

Department of Computer Science & Engineering (IoT)

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.

The screenshot displays a web application interface for a smart contract. On the left, there is a list of functions: **transferOwnership** (address), **updateStudent** (uint256, string), **getStudent** (uint256), and **isRegistered** (address). Each function has a 'Transact' button. Below the functions, there is a 'Call' button for the **getStudent** function. The right panel shows the transaction details for the selected function. It includes a search bar for transaction hash or address, and a table with columns for output, decoded input, decoded output, logs, and raw logs. The bottom panel shows the **registeredFunction** function with a 'Call' button and a 'Low level interaction' section.



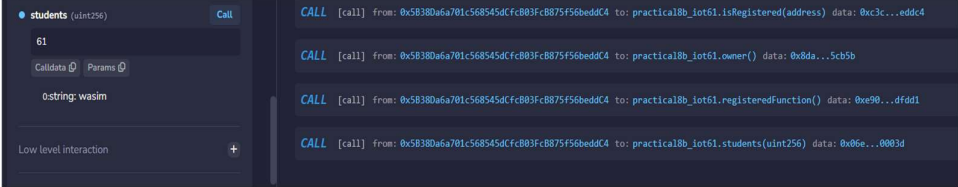
Department of Computer Science & Engineering (IoT)

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.

	
Code	<pre>// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract practical8_iot61 { mapping(uint => string) public students; mapping(address => bool) public isRegistered; address public owner; constructor() { owner = msg.sender; } modifier onlyOwner() { require(msg.sender == owner, "Access Denied: Not Owner"); _; } modifier onlyRegistered() { require(isRegistered[msg.sender], "User not registered"); _; } function addstudent(uint _id, string memory _name) public onlyOwner { students[_id] = _name; } function getstudent(uint _id) public view returns (string memory) { return students[_id]; } }</pre>

**Department of Computer Science & Engineering (IoT)****Vision of the Department***To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.***Mission of the Department***To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.*

	<pre>} function updateStudent(uint _id, string memory _name) public onlyOwner { students[_id] = _name; } function registerUser(address _user) public onlyOwner { isRegistered[_user] = true; } function registeredFunction() public view onlyRegistered returns (string memory) { return "You are a registered user"; } function transferOwnership(address newOwner) public onlyOwner { require(newOwner != address(0), "Invalid new owner"); owner = newOwner; } }</pre>
Conclusion	The smart contract was successfully implemented using mappings and modifiers in Solidity. Mappings were used to store student information and user registration status, while modifiers were used to provide access control. The onlyOwner modifier ensures that only the contract owner can add, update student records, and transfer ownership. The registration mechanism allows user addresses to be recorded on the blockchain. Thus, the practical demonstrates how Solidity can be used to create a secure and controlled smart contract for managing data on a blockchain.
Date	15/09/2026